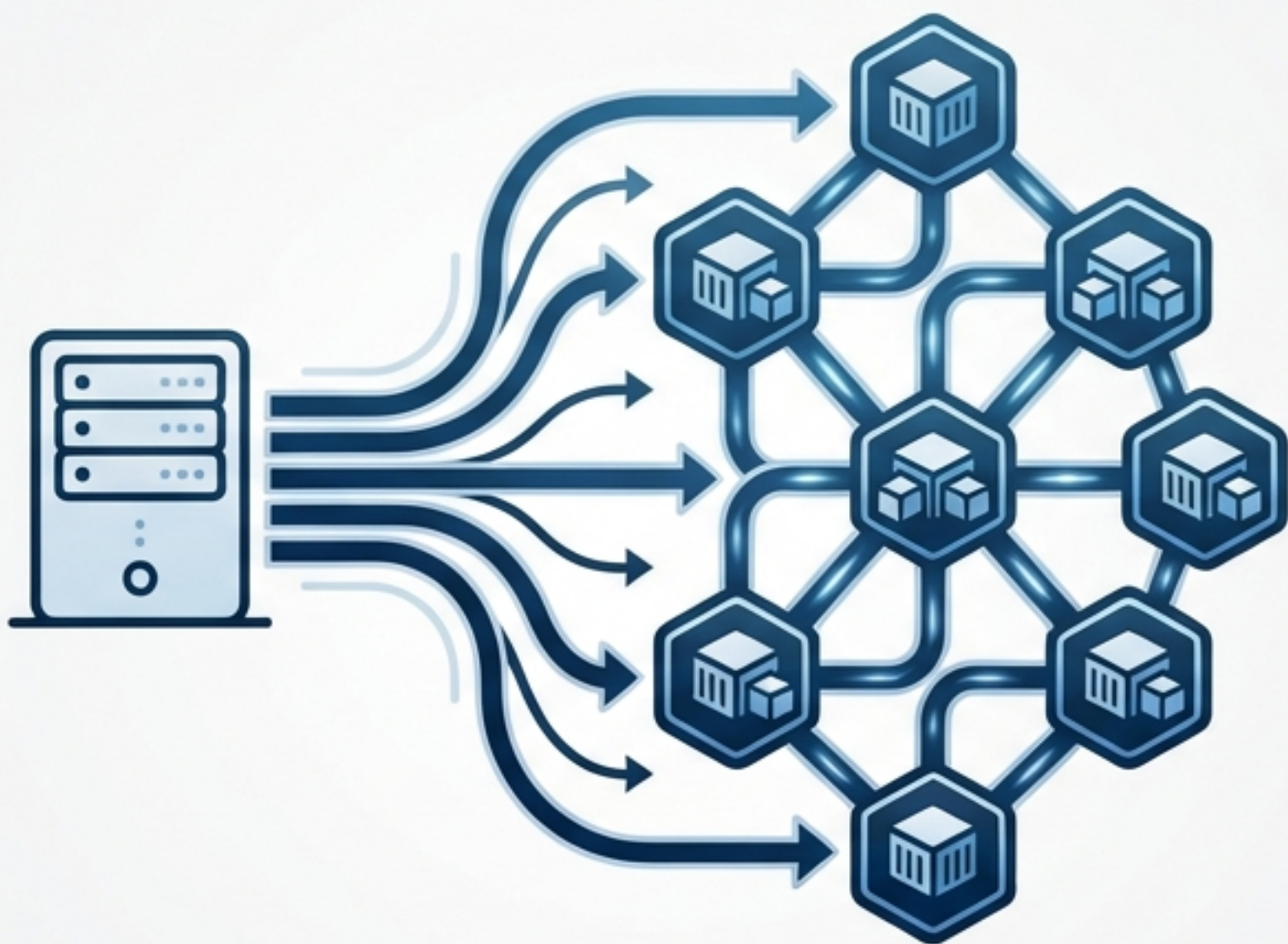




# Docker Swarm: De Máquina Única a um Exército Coordenado

A sua jornada para a orquestração de contêineres distribuídos.

Nesta apresentação, vamos migrar do gerenciamento local de contêineres para a orquestração distribuída em cluster. Enquanto o Docker Compose nos ajuda a organizar múltiplos contêineres em uma única máquina, o Docker Swarm nos permite distribuir e gerenciar aplicações através de múltiplos hosts. Prepare-se para entender como transformar um conjunto de servidores em um recurso único e poderoso, capaz de orquestrar centenas de contêineres de forma inteligente e resiliente.



# O Desafio: Por Que Precisamos de Orquestração?

## Definição de Orquestração de Cluster

Orquestração de cluster é o processo de gerenciar a implantação, escalabilidade, rede e ciclo de vida de contêineres distribuídos em múltiplos hosts. Em vez de gerenciar cada servidor individualmente, você trata todo o cluster como um **único recurso lógico**, onde o orquestrador decide automaticamente onde executar cada contêiner.

## Composição de um Cluster

Um cluster é formado por um **grupo de máquinas** que trabalham juntas:

- Podem ser servidores físicos, VMs ou instâncias na nuvem.
- Comunicam-se através de uma rede compartilhada.
- Funcionam como uma única unidade de computação.
- Distribuem a carga de trabalho automaticamente.





# A Anatomia do Swarm: O Cérebro e os Músculos

## Managers (O Cérebro do Cluster)

Responsabilidades:

- Gerenciar o estado do cluster.
- Distribuir tarefas aos Workers.
- Manter consistência do estado desejado.
- Tomar decisões de orquestração.

**Redundância:** Recomenda-se 3 ou 5 Managers para garantir alta disponibilidade (algoritmo de consenso Raft).



## Workers (A Força de Execução)

Responsabilidades:

- Executam as tarefas (contêineres) enviadas pelos Managers.
- Reportam status de saúde aos Managers.
- Não tomam decisões de orquestração.
- Podem ser promovidos a Managers se necessário.

**Escalabilidade:** Adicione quantos Workers forem necessários para sua carga de trabalho.



# O Plano Mestre: Entendendo o Objeto "Service"



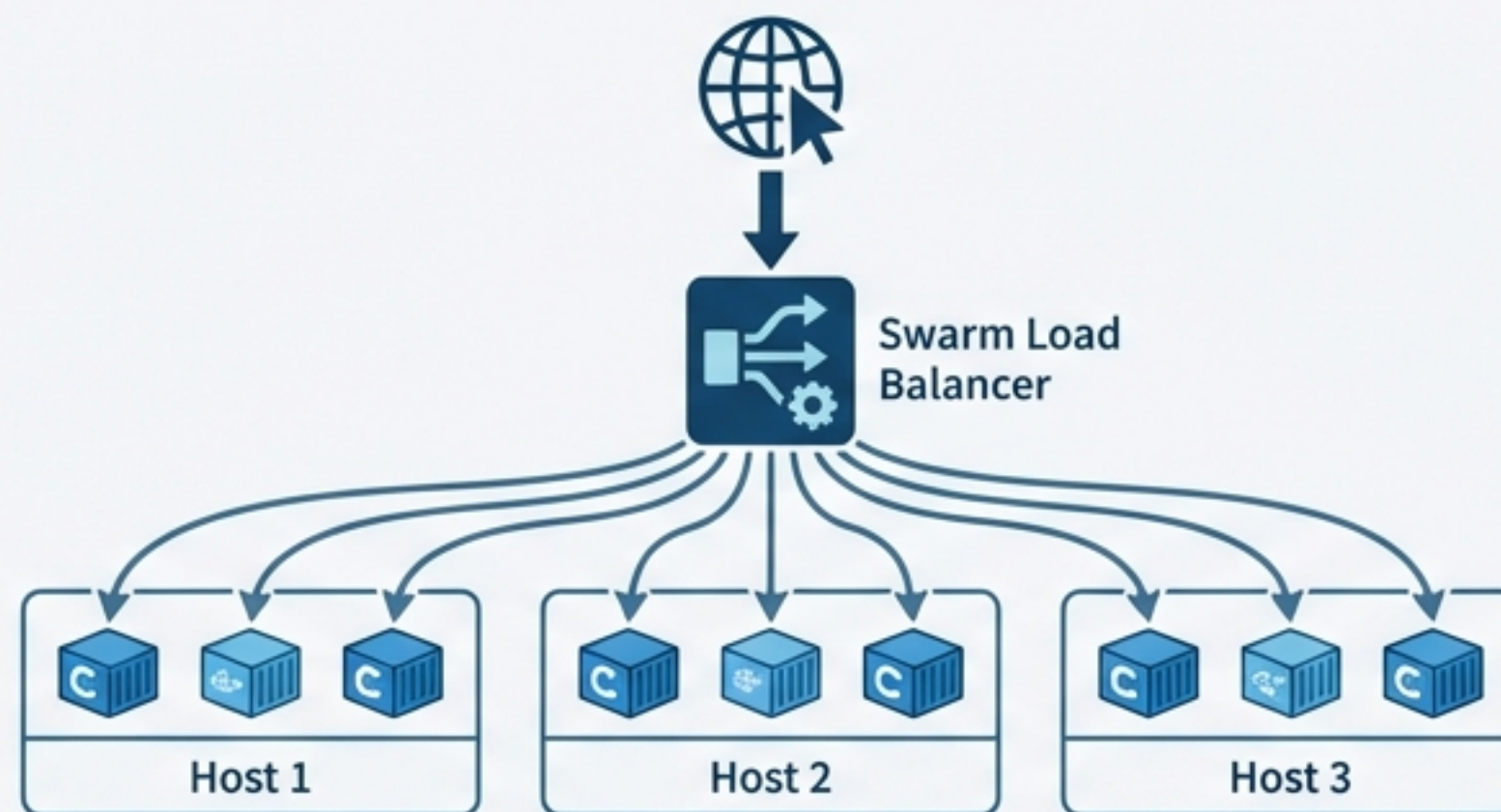
O **Service** é o conceito fundamental no Docker Swarm. Ele representa a **descrição do estado desejado** da sua aplicação. Por exemplo: "Quero 5 réplicas da imagem Nginx rodando na porta 80".

Uma **réplica** é cada instância individual de contêiner que executa o Service. O Swarm garante que o número de réplicas em execução sempre corresponda ao estado desejado.

O comando principal para criar um Service é `docker service create`, substituindo o `docker run` que usávamos para contêineres isolados.



# Superpoder #1: Escalabilidade Simples e Balanceamento de Carga Automático



## Escalabilidade Declarativa

Alterar a capacidade do seu serviço é tão simples quanto modificar um número.

```
docker service scale nome_servico=10
```

O Swarm ajusta automaticamente para 10 réplicas.

**O processo é declarativo:** você define o estado desejado e o Swarm cuida da implementação.

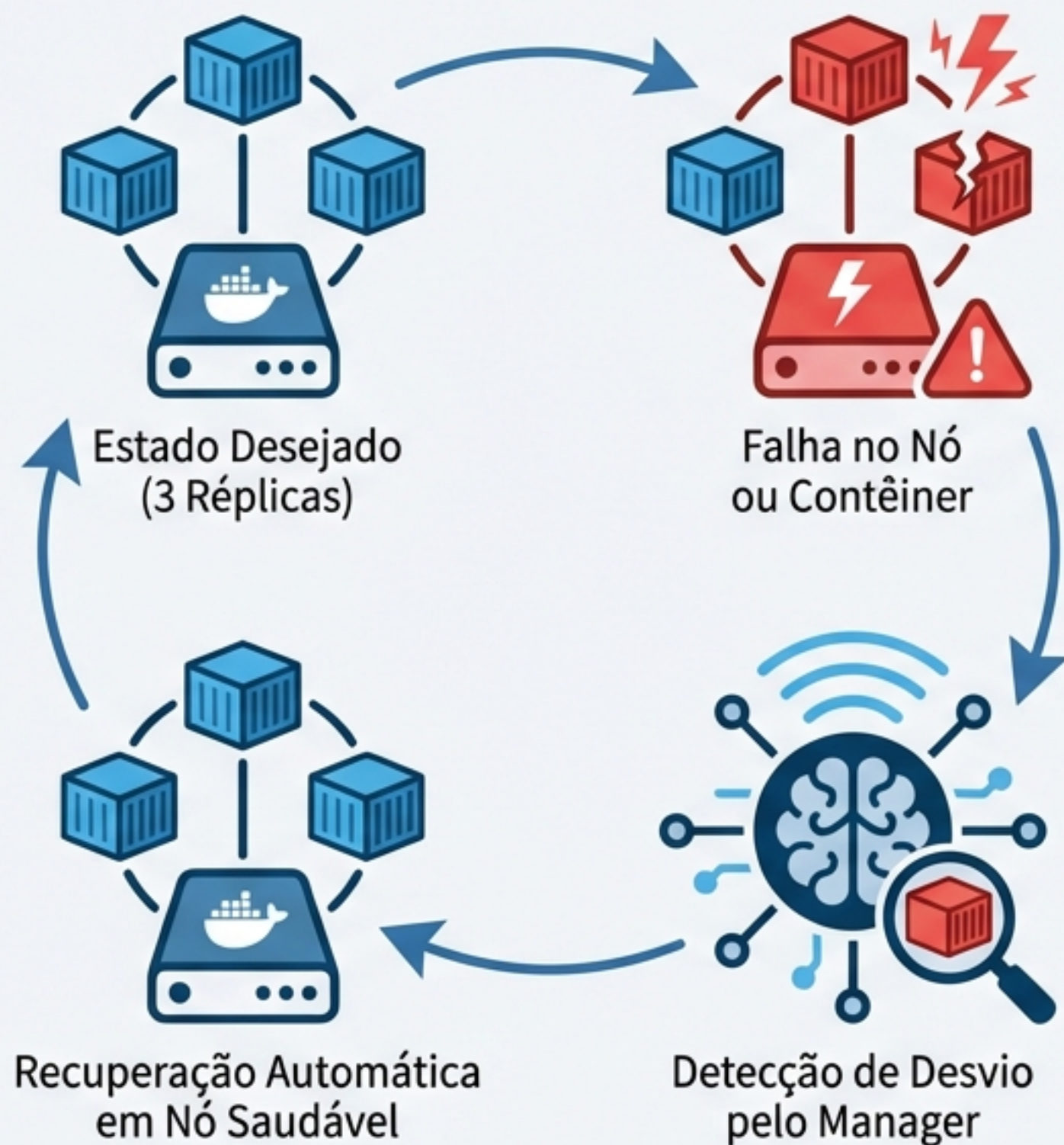
## Load Balancing Integrado

O Swarm distribui automaticamente o tráfego de entrada para todas as réplicas ativas, **independentemente do host onde estão executando.**

Utiliza a **rede Overlay** para roteamento inteligente, garantindo distribuição equilibrada de carga entre todos os contêineres disponíveis.



# Superpoder #2: Tolerância a Falhas com Autorrecuperação Inteligente



O Docker Swarm monitora continuamente a saúde do cluster. Se um **nó falhar** ou um **contêiner morrer**, os Managers detectam imediatamente o desvio do estado desejado.

## Ação Automática (Sem intervenção manual):

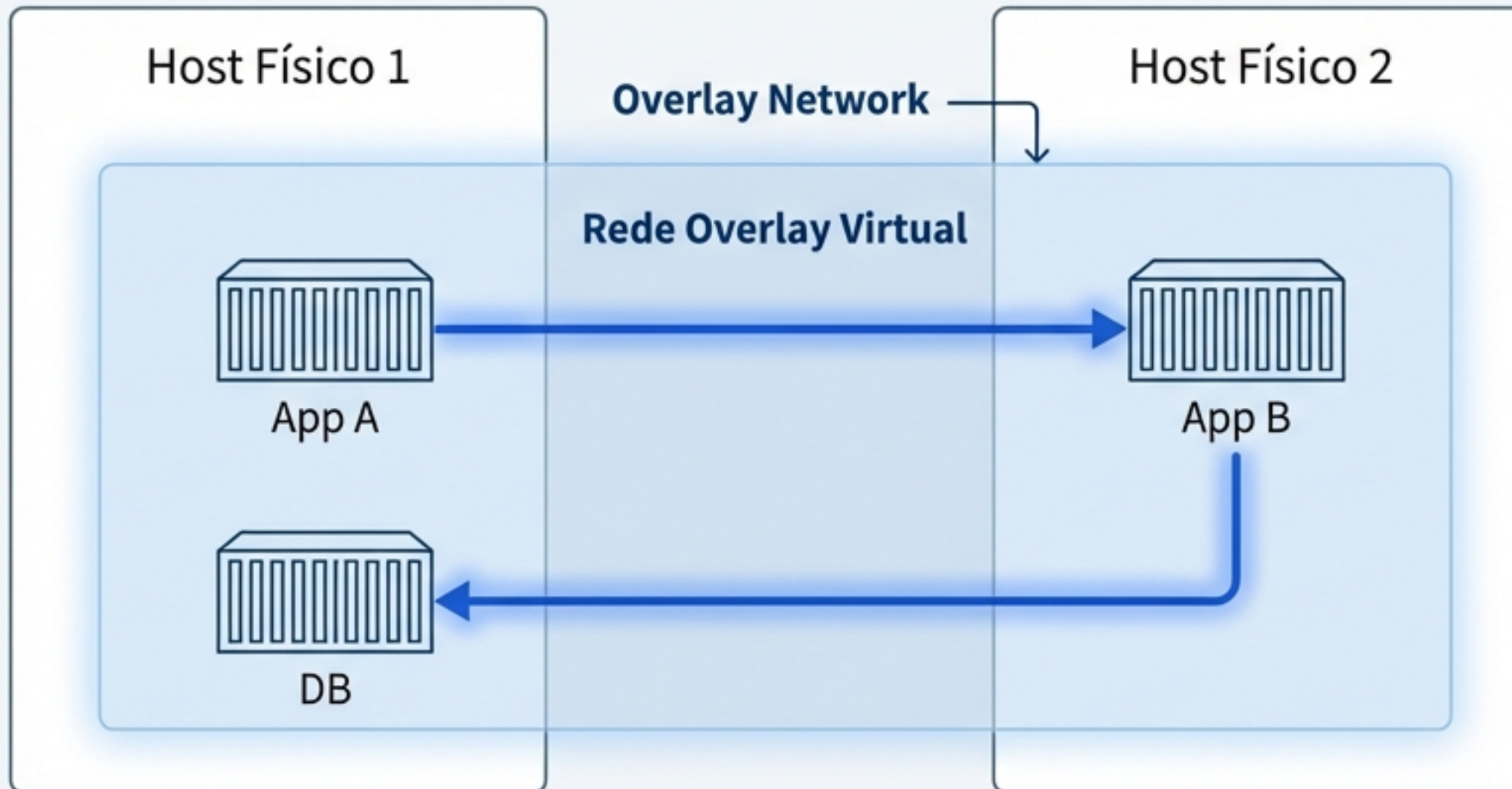
- 1** O Manager identifica réplicas perdidas ou não-saudáveis.
- 2** Seleciona nós saudáveis disponíveis.
- 3** Relança os contêineres necessários.
- 4** Restaura o número de réplicas configurado.

**Resultado:** Isso garante que sua aplicação mantenha a **disponibilidade desejada**, mesmo diante de falhas de hardware ou software.



# Superpoder #3: A Rede Overlay e o Poder da Comunicação

## I o Overlay Network



### Driver Overlay

O driver Overlay é a rede especial usada pelo Swarm para comunicação entre hosts diferentes. Ela cria uma camada de rede virtual que conecta contêineres em máquinas distintas.

### Comunicação Transparente

Com a rede Overlay, contêineres em diferentes máquinas se comportam **como se estivessem na mesma rede local**. Isso simplifica drasticamente a arquitetura de microsserviços distribuídos.

### Service Discovery Integrado

A rede Overlay possibilita o **Service Discovery automático** e o **Load Balancing interno**, permitindo que serviços encontrem e se comuniquem entre si usando apenas nomes de serviço.



# Sua Primeira Missão: Lançando um Serviço Web

## 1. Inicializar o Swarm

```
docker swarm init
```

Transforma a máquina atual no primeiro Manager do cluster.

## 3. Verificar o Status

```
docker service ls  
docker service ps web-app
```

Confirma que o serviço está rodando e visualiza a distribuição das réplicas.

## 2. Criar o Serviço

```
docker service create \  
  --name web-app \  
  --replicas 3 \  
  --publish 80:80 \  
  nginx:latest
```

Cria um serviço com 3 réplicas do Nginx acessíveis na porta 80.

## 4. Escalar o Serviço

```
docker service scale web-app=10
```

Aumenta para 10 réplicas instantaneamente, distribuídas automaticamente.



# O Arsenal do Orquestrador: Comandos Essenciais

## Inicialização e Adesão ao Cluster

```
docker swarm init  
docker swarm join --token [TOKEN] [IP:PORT]
```

*Inicialize o Swarm no primeiro Manager e adicione outros nós usando o token gerado.*

## Gerenciamento de Nós

```
docker node ls  
docker node inspect [NODE_ID]
```

*Visualize o status de todos os nós do cluster e inspecione detalhes específicos.*

## Gerenciamento de Serviços

```
docker service create --name web --replicas 3 nginx  
docker service scale web=5  
docker service rm web
```

*Crie, escale e remova serviços de forma declarativa.*

## Monitoramento de Réplicas

```
docker service ps [SERVICE_NAME]  
docker service logs [SERVICE_NAME]
```

*Monitore o status e logs de todas as réplicas do serviço.*



# Onde o Swarm Brilha: Cenários de Uso Ideais



## **Startups e MVPs**

Perfeito para startups que precisam implantar rapidamente sem a complexidade do Kubernetes. Foco no produto enquanto garante escalabilidade básica.



## **Ambientes de Desenvolvimento**

Ideal para replicar ambientes de produção em desenvolvimento e staging, permitindo testes realistas de aplicações distribuídas.



## **Aprendizado de Orquestração**

Excelente ponto de partida para estudantes e profissionais aprenderem conceitos de orquestração antes de avançar para sistemas mais complexos.



## **Projetos de Médio Porte**

Adequado para aplicações que precisam de alta disponibilidade e escalabilidade, mas não justificam a complexidade operacional do Kubernetes.



# As Regras do Jogo: Boas Práticas para o Sucesso

1

## Sempre Use Múltiplos Managers

Mantenha 3 ou 5 Managers (número ímpar) para garantir alta disponibilidade e consenso adequado. Um único Manager é ponto único de falha.

2

## Dimensione Réplicas Adequadamente

Configure réplicas baseadas na carga esperada e distribua-as entre múltiplos nós para garantir redundância e performance.

3

## Use Secrets para Dados Sensíveis

Nunca armazene senhas ou chaves em variáveis de ambiente. Use ``docker secret create`` para gerenciar dados sensíveis com segurança.

4

## Monitore Continuamente

Implemente monitoramento de logs, métricas e health checks para detectar problemas antes que afetem os usuários.

5

## Documente a Topologia

Mantenha documentação clara da arquitetura do cluster, incluindo papel de cada nó e dependências entre serviços.



# Conhecendo os Limites: Vantagens do Swarm e a Transição

## Vantagens do Swarm

- + **Simplicidade:** Fácil de configurar e usar.
- + **Nativo do Docker:** Integração perfeita.
- + **Curva de aprendizado:** Suave para iniciantes.
- + **Overhead baixo:** Menos recursos necessários.
- + **Ideal para:** Projetos pequenos a médios.

## Limitações Comparadas ao K8s

- **Recursos de rede:** Menos flexíveis.
- **Storage complexo:** Opções limitadas.
- **Ecossistema:** Menor que Kubernetes.
- **Customização:** Menos configurável.
- **Escala empresarial:** K8s é preferido.

### O Swarm como ponte:

O Docker Swarm é a ponte perfeita para entender a necessidade e a lógica de orquestração antes de migrar para o Kubernetes. Dominar o Swarm facilita significativamente o aprendizado do K8s.



# O Duelo de Titãs: Docker Swarm vs. Kubernetes

| Característica | Docker Swarm             | Kubernetes                       |
|----------------|--------------------------|----------------------------------|
| Complexidade   | Simples e direto         | Complexo, mais recursos          |
| Instalação     | Nativa do Docker         | Requer setup adicional           |
| Escalabilidade | Até centenas de nós      | Milhares de nós                  |
| Ecosistema     | Limitado                 | Vasto e maduro                   |
| Rede           | Overlay básica           | Plugins e políticas avançadas    |
| Storage        | Volumes Docker           | Múltiplos backends               |
| Monitoramento  | Básico                   | Extensivo com Prometheus/Grafana |
| Adoção         | Projetos pequenos/médios | Padrão da indústria              |



# A Jornada Continua: Próximos Passos Rumo ao Kubernetes

## Base Estabelecida

Você agora compreende os conceitos fundamentais de orquestração: clusters, réplicas, serviços, escalabilidade e tolerância a falhas.

## A Próxima Etapa: Kubernetes

Na próxima aula, vamos explorar o Kubernetes (K8s), a plataforma de orquestração de nível industrial. Aprenderemos sobre Pods, Deployments e Services.

## Evolução Profissional

O conhecimento de orquestração é essencial para DevOps e Cloud Native. Você está construindo habilidades altamente valorizadas no mercado.



*"A jornada de mil milhas começa com um único passo. O Swarm foi seu primeiro passo na orquestração — o Kubernetes será o próximo grande salto!"*